UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026).
   Answers are unofficial reference answers, not official UTM marking schemes.

2. Marking points show the expected answer and mark allocation; check them against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Approved real questions only; no generators, placeholders or parametric variants.
   Questions and answers are for local study use only.

5. Structure derived from the supplied CSC429 papers; every question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 9 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION 1 (5 marks)

Source: CSC429-2019-12, page [2]

Unofficial model answer:

a) The Von Neumann Architecture was first introduced in the **first generation** of computers (the stored-program concept, associated with the EDVAC design, c. 1945–1950s).

b) A labelled Von Neumann Architecture diagram should show the CPU (containing the Control Unit (CU), Arithmetic Logic Unit (ALU), and registers) connected to Main Memory, with Input and Output devices, all linked by a common bus/system bus. Data and instructions share the same memory and bus.

c) The main task of the Control Unit (CU) is to **fetch, decode and direct the execution of instructions**. It fetches instructions from main memory, decodes them, and generates the control signals that coordinate the ALU, registers, memory and I/O so that each instruction is carried out in the correct sequence. It does not itself perform arithmetic; it controls and synchronises the other components.

Marking points:

• *1 mark: (a) First generation (stored-program concept).*

• *2 marks: (b) Correct diagram with CPU (CU + ALU + registers), main memory, input and output, connected by a common bus (1 mark for components, 1 mark for correct connections/labels).*

• *2 marks: (c) CU fetches and decodes instructions and generates control signals to coordinate/direct other components (1 mark for fetch/decode, 1 mark for control/coordination of execution).*

Model confidence: high.

Model note: Part (b) is a drawing question; the answer describes the required diagram since no figure is supplied. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 2 (5 marks)

Source: CSC429-2023-02, page [2]

Unofficial model answer:

a) Mechanical Calculating Machine: Blaise Pascal (Pascaline, 1642). b) Difference Machine: Charles Babbage (Difference Engine, 1822). c) Analytical Machine: Charles Babbage (Analytical Engine, 1830s). d) IAS Machine: John von Neumann (IAS computer, Institute for Advanced Study, Princeton, 1945). e) Integrated circuit: Jack Kilby (1958, Texas Instruments); Robert Noyce (1959, Fairchild) independently developed a silicon IC, so either name is acceptable, with Kilby being the standard textbook answer.

Marking points:

• *1 mark: a) Blaise Pascal*

• *1 mark: b) Charles Babbage*

• *1 mark: c) Charles Babbage*

• *1 mark: d) John von Neumann*

• *1 mark: e) Jack Kilby (accept Robert Noyce)*

Model confidence: high.

Model note: Unofficial model answer, not an official UTM marking scheme. For (e), some texts credit Noyce as co-inventor; accept either. The question wording 'Difference Machine' and 'Analytical Machine' is taken to mean Babbage's Difference Engine and Analytical Engine.

QUESTION 3 (5 marks)

Source: CSC429-2014-12, page [3]

© Hak Cipta Universiti Teknologi MARA

Unofficial model answer:

a) Speed: SRAM is faster than DRAM. SRAM access times are typically on the order of a few nanoseconds (e.g., 1–10 ns), while DRAM access times are longer (e.g., 10–100 ns) due to the need to sense and restore charge. SRAM is used for cache memory; DRAM for main memory.

b) Size per bit: SRAM uses more transistors per bit (typically 6 transistors per bit, forming a flip-flop), so it occupies a larger chip area per bit. DRAM uses one transistor plus one capacitor per bit, giving a much smaller size per bit and higher density.

c) Cost: SRAM is more expensive per bit than DRAM because of its larger cell size and lower density. DRAM is cheaper per bit and is therefore used for large-capacity main memory.

d) Type of bits stored: SRAM stores bits as the state of a bistable flip-flop (voltage levels held by cross-coupled inverters), so the stored value is retained as long as power is applied. DRAM stores bits as electric charge on a capacitor; the presence or absence of charge represents the bit value.

e) Refreshing need when powered: SRAM does not require refreshing while powered; the flip-flop holds its state indefinitely as long as power is supplied. DRAM requires periodic refreshing (typically every few milliseconds) because the capacitor charge leaks away, even when powered.

Marking points:

• 1 mark: Speed – SRAM faster than DRAM (with brief reason such as no refresh/sense-restore overhead).

• 1 mark: Size per bit – SRAM larger cell (e.g., 6 transistors) vs DRAM smaller cell (1 transistor + 1 capacitor).

• 1 mark: Cost – SRAM more expensive per bit than DRAM.

• 1 mark: Type of bits stored – SRAM uses flip-flop/latch state; DRAM uses capacitor charge.

• 1 mark: Refreshing need when powered – SRAM no refresh needed; DRAM needs periodic refresh.

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. No figure or table was supplied. Typical transistor counts and refresh intervals are illustrative and not required for full marks.

QUESTION 4 (5 marks)

Source: CSC429-2018-06, page [4]

Unofficial model answer:

a) A cache is a small, fast memory that holds copies of frequently accessed data and instructions from main memory, placed between the CPU and main memory to reduce average memory access time. It exploits locality of reference (temporal and spatial) so that most accesses hit in the cache rather than going to slower main memory.

b) Two ways to store data in cache: (1) Direct-mapped cache: each main-memory block maps to exactly one cache line, determined by block address modulo number of lines; simple and fast but can suffer conflict misses. (2) Set-associative cache: the cache is divided into sets, each containing several lines; a block maps to a specific set but can occupy any line within that set, reducing conflict misses at the cost of extra comparison logic. (Fully associative, where a block can go in any line, is an alternative acceptable second way.)

Marking points:

• 1 mark: cache is small/fast memory between CPU and main memory

• 1 mark: holds copies of frequently used data/instructions and exploits locality to reduce average access time

• 1 mark: direct-mapped — each block maps to exactly one cache line (index = block address mod number of lines)

• 1 mark: set-associative — cache divided into sets, block maps to a set and can occupy any line in that set

• 1 mark: correct contrast/benefit (e.g., direct-mapped simple/fast but conflict misses; set-associative reduces conflict misses) or fully associative described correctly

Model confidence: high.

Model note: Unofficial model answer, not an official UTM marking scheme. No figure/table was supplied; answer is based on

© Hak Cipta Universiti Teknologi MARA

standard CSC429 cache terminology.

QUESTION 5 (10 marks)

Source: CSC429-2018-06, page [5]

Unofficial model answer:

a) Three types of interfaces to external devices: (1) Programmed I/O (polling) interface: the CPU repeatedly checks the status register of the device controller and transfers data through the data register when the device is ready; no interrupt is used, so the CPU wastes cycles waiting. (2) Interrupt-driven I/O interface: the device controller raises an interrupt request when it has completed a transfer or is ready; the CPU suspends the current program, runs an interrupt service routine to service the device, then resumes, freeing the CPU from polling. (3) Direct Memory Access (DMA) interface: a DMA controller is programmed with the memory address, count and direction, and it transfers blocks of data directly between the device and main memory without CPU involvement, issuing an interrupt only when the whole block is complete. (Other acceptable interfaces: memory-mapped I/O vs isolated I/O addressing, or serial/parallel/USB interfaces, if described as interfaces to external devices.)

b) Two types of interrupt: (1) Maskable interrupt: an interrupt that can be temporarily disabled (masked) by the CPU by clearing the interrupt-enable flag or by an interrupt mask register; it is used for ordinary I/O devices and is checked after the current instruction. (2) Non-maskable interrupt (NMI): an interrupt that cannot be disabled by the CPU; it is reserved for critical events such as power failure or hardware fault and is always serviced. (Alternatively, classified by source: hardware/external interrupt vs software/trap interrupt; or by timing: synchronous vs asynchronous.)

Marking points:

- *1 mark: correct identification of programmed I/O (polling) interface with brief description*
- *1 mark: correct identification of interrupt-driven I/O interface with brief description*
- *1 mark: correct identification of DMA interface with brief description*
- *1 mark: accurate explanation of maskable interrupt (can be disabled/masked)*
- *1 mark: accurate explanation of non-maskable interrupt (cannot be disabled, critical use)*
- *1 mark: clear terminology and coherent explanation for each part*
- *1 mark: each type of interface linked to how CPU/device interaction occurs*
- *1 mark: each interrupt type linked to its purpose or handling*
- *1 mark: answer covers all three interfaces and two interrupts without omission*
- *1 mark: overall clarity and correctness (no major conceptual error)*

Model confidence: high.

Model note: No figure or table was supplied; the answer is based on standard computer-organization content. The question wording 'types of interfaces to external devices' is interpreted as I/O techniques (programmed, interrupt-driven, DMA); alternative valid interpretations (e.g., memory-mapped vs isolated I/O, serial vs parallel) are acknowledged in the answer. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 6 (10 marks)

Source: CSC429-2014-12, page [5]

Unofficial model answer:

Multiple-processor architectures are commonly classified by how processors share memory and how they are interconnected. Four standard classifications, each with an example, are: (1) Shared-memory multiprocessor (tightly coupled / UMA): all processors access a single global physical memory through a common bus or crossbar, so inter-processor communication is via shared variables in memory; example: a dual-socket Intel Xeon server board with two CPUs sharing main memory. (2) Message-passing multicomputer (loosely coupled / distributed memory): each processor has its own private local memory and processors communicate by

© Hak Cipta Universiti Teknologi MARA

explicitly sending and receiving messages over an interconnection network; example: a Beowulf cluster of PCs connected by Ethernet, or an IBM Blue Gene supercomputer. (3) SIMD (Single Instruction, Multiple Data) array processor: one control unit issues a single instruction that operates simultaneously on many processing elements, each working on a different data element; example: a GPU's SIMT/vector units or a classic array processor such as the ILLIAC IV. (4) NUMA (Non-Uniform Memory Access) multiprocessor: a shared address space is physically distributed among nodes, so a processor's access time depends on whether the data is in its local node's memory or a remote node's memory; example: an SGI Origin or a modern AMD EPYC multi-socket server. (Acceptable alternatives: MIMD vs SIMD vs MISD vs SISD Flynn classification; master-slave vs symmetric multiprocessing; UMA vs NUMA vs COMA; vector processors; systolic arrays.)

Marking points:

• 1 mark: correctly naming/defining classification 1 (e.g., shared-memory UMA multiprocessor) with a valid example

• 1 mark: correctly naming/defining classification 2 (e.g., message-passing distributed-memory multicomputer) with a valid example

• 1 mark: correctly naming/defining classification 3 (e.g., SIMD array/vector processor) with a valid example

• 1 mark: correctly naming/defining classification 4 (e.g., NUMA multiprocessor) with a valid example

• 2 marks: accuracy and appropriate use of terminology (tightly/loosely coupled, UMA/NUMA, SIMD, message passing, interconnection network) across the four descriptions

• 2 marks: examples are technically correct and clearly matched to each classification

• 2 marks: overall coherence, clarity and completeness of the four descriptions (each classification explained, not merely named)

Model confidence: high.

Model note: The question text does not specify a particular taxonomy (e.g., Flynn's or Tanenbaum's), so the answer uses the common memory-coupling classification and notes Flynn-style alternatives as acceptable. No figure or table was supplied; if the original paper expected a specific four-way scheme, marks should be mapped to that scheme. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 7 (10 marks)

Source: CSC429-2016-12, page [3]

Unofficial model answer:

a) Architecture vs organization: Computer architecture refers to the attributes of a computer system that are visible to a programmer, i.e., the conceptual design and functional behaviour of the system. It includes the instruction set architecture (ISA), data types, addressing modes, registers visible to the programmer, and I/O mechanisms. Architecture answers 'what the computer does' from the programmer's perspective. Computer organization refers to the operational units and their interconnections that realize the architectural specifications. It deals with the internal hardware details that are transparent to the programmer, such as control signals, memory technology, bus structure, and the physical implementation of the CPU. Organization answers 'how the computer implements the architecture'. For example, whether a computer has a multiply instruction is an architectural issue, while whether that multiply is implemented by a dedicated hardware multiplier or by repeated addition is an organizational issue. Another example: the programmer-visible register set is architecture, while the technology and arrangement of the ALU and control unit are organization. In short, architecture is the programmer-visible functional view, and organization is the implementation-level structural view.

b) Three CPU components and their functions: (1) Control Unit (CU): The control unit directs the operation of the CPU and the rest of the computer. It fetches and decodes instructions, then generates the timing and control signals that cause the other components (ALU, registers, memory, I/O) to perform the required operations. It coordinates the fetch-decode-execute cycle and ensures that data moves correctly between units. (2) Arithmetic Logic Unit (ALU): The ALU performs arithmetic operations such as addition, subtraction, multiplication, and division, and logical operations such as AND, OR, NOT, XOR, as well as comparisons and

© Hak Cipta Universiti Teknologi MARA

shifts. It takes operands from registers, performs the requested operation, and places the result in a register while setting condition flags (e.g., zero, carry, sign, overflow). (3) Registers: Registers are small, high-speed storage locations inside the CPU. They hold operands, results, addresses, and control information during instruction execution. Key registers include the Program Counter (PC), which holds the address of the next instruction; the Instruction Register (IR), which holds the current instruction being executed; the Memory Address Register (MAR) and Memory Data Register (MDR) for memory transfers; and general-purpose registers for temporary data. Registers provide the fastest storage in the memory hierarchy and reduce the number of slow memory accesses. (Other acceptable components include the internal buses, cache, and instruction decoder, but the three above are the standard expected answers.)

Marking points:

• 1 mark: Correct definition of architecture as programmer-visible attributes (ISA, data types, addressing modes, etc.).

• 1 mark: Correct definition of organization as implementation-level structure (control signals, buses, technology, etc.) that realizes the architecture.

• 1 mark: Clear distinction with at least one example (e.g., multiply instruction vs hardware multiplier).

• 1 mark: Identification of Control Unit (CU) as a CPU component.

• 1 mark: Explanation of CU function (fetch/decode, generate control signals, coordinate execution).

• 1 mark: Identification of Arithmetic Logic Unit (ALU) as a CPU component.

• 1 mark: Explanation of ALU function (arithmetic and logical operations, flags).

• 1 mark: Identification of registers (or another valid CPU component) as a CPU component.

• 1 mark: Explanation of register function (high-speed temporary storage, PC/IR/MAR/MDR roles).

• 1 mark: Overall clarity, correct terminology, and completeness of the answer.

Model confidence: high.

Model note: This is an unofficial model answer, not an official UTM marking scheme. The question does not specify which three CPU components to discuss; the answer uses the Control Unit, ALU, and registers, which are the most standard choices. Other valid components (e.g., instruction decoder, internal buses, cache) could be accepted if explained correctly. Mark allocation is indicative and assumes a 10-mark total with roughly 4 marks for part (a) and 6 marks for part (b).

QUESTION 8 (10 marks)

Source: CSC429-2025-07, page [3]

Unofficial model answer:

a) Convert 46.0703125 to IEEE-754 single precision (hex). Step 1 — sign: number is positive, so S = 0. Step 2 — normalize: 46.0703125 = 101110.0001001_2 (46 = 101110_2; 0.0703125 = 0.0001001_2 since 0.0703125 = 9/128 = 1/16 + 1/128). Normalize: 1.011100001001_2 × 2^5. Step 3 — exponent: E = 5 + 127 = 132 = 10000100_2. Step 4 — mantissa (fraction after leading 1): 011100001001, pad to 23 bits: 0111 0000 1001 0000 0000 000. Step 5 — assemble: 0 10000100 01110000100100000000000. Group into nibbles: 0100 0010 0011 1000 0100 1000 0000 0000 = 0x42384800. Answer: 0x42384800.

b) Convert 0x8F080000 to decimal. Step 1 — binary: 1000 1111 0000 1000 0000 0000 0000 0000. Step 2 — fields: S = 1 (negative); exponent = 0001 1110 = 30; fraction = 000 1000 0000 0000 0000 0000. Step 3 — unbiased exponent: 30 − 127 = −97. Step 4 — significand: 1.0001_2 = 1 + 1/16 = 1.0625. Step 5 — value = −1.0625 × 2^−97 ≈ −6.71 × 10^−30 (exact value −1.0625 × 2^−97). Answer: approximately −6.71 × 10^−30.

Marking points:

• a) 1 mark: correct integer/fraction binary conversion (101110.0001001)

• a) 1 mark: correct normalization 1.011100001001 × 2^5

• a) 1 mark: biased exponent 132 = 10000100

• a) 1 mark: correct 23-bit mantissa field

• a) 1 mark: correct final hex 0x42384800 with sign bit

• b) 1 mark: correct binary expansion of 8F080000

© Hak Cipta Universiti Teknologi MARA

• b) 1 mark: correct field split S=1, E=30, fraction

• b) 1 mark: unbiased exponent −97 and significand 1.0625

• b) 1 mark: correct decimal ≈ −6.71 × 10^−30 (accept exact form)

• b) 1 mark: clear workflow shown for both parts

Model confidence: high.

Model note: Question text has OCR artefacts: '46.0703'125n' read as 46.0703125 and '8F080000rr' read as 0x8F080000. If the intended values differ, the method still applies. No figure supplied.

QUESTION 9 (10 marks)

Source: CSC429-2023-07, page [4]

Unofficial model answer:

a) An interrupt is a signal (hardware or software) sent to the CPU that temporarily suspends the currently executing program, causes the CPU to save its context (program counter and register state), and transfers control to a dedicated interrupt handler/service routine. After the handler completes, the CPU restores the saved context and resumes the interrupted program. Interrupts allow the processor to respond promptly to asynchronous events (I/O completion, timers, errors) without polling.

b) Based on Figure 1, the interrupts are ranked by priority from most to least important as: Interrupt Handler W > Interrupt Handler X > Interrupt Handler Y > Interrupt Handler Z. This ordering follows the nested/multiple-interrupt sequence shown in the figure, where a higher-priority interrupt preempts a lower-priority one; the handler entered first and preempted by later handlers (W) is the highest priority, and the handler that is entered last and completes first (Z) is the lowest priority.

c) If interrupts are disabled (interrupt disable applied), preemption cannot occur, so the handlers execute to completion in the order they are first entered, i.e. non-preemptive/sequential execution. In that case the ranking from most to least important becomes: Interrupt Handler W > Interrupt Handler X > Interrupt Handler Y > Interrupt Handler Z, because the first interrupt accepted (W) is serviced fully before the next (X), then Y, then Z; disabling interrupts removes the nesting/preemption that would otherwise reorder completion.

Marking points:

• 1 mark: correct definition of interrupt (signal to CPU that suspends current program and transfers control to a handler, with context save/restore).

• 1 mark: mention of saving/restoring CPU context or resuming the interrupted program.

• 2 marks: correct priority ranking W > X > Y > Z for part (b).

• 1 mark: justification that higher-priority interrupt preempts lower-priority (nested interrupts).

• 2 marks: correct ranking for part (c) under disabled interrupts.

• 1 mark: explanation that disabling interrupts prevents preemption, giving sequential/non-preemptive service.

• 2 marks: coherent overall explanation linking priority, nesting, and interrupt disable to the figure.

Model confidence: medium.

Model note: Figure 1 is supplied only as a caption/crop description; the exact nesting arrows are not fully visible in the extracted text. The ranking assumes the standard textbook multiple-interrupt diagram where W is entered first and preempted by X, Y, Z in that order. If the actual figure shows a different nesting order, the ranking letters should be adjusted accordingly. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 10 (10 marks)

Source: CSC429-2023-02, page [3]

Unofficial model answer:

In a typical computer system, registers are small, fast storage locations inside the CPU used to hold data, addresses, and control information during program execution. The main registers and their functions are: (1) Program Counter (PC) / Instruction Pointer: holds the address of the next instruction to be fetched; it is

© Hak Cipta Universiti Teknologi MARA

automatically incremented or updated after each fetch so the CPU knows where to get the next instruction. (2) Instruction Register (IR): holds the instruction currently being fetched/decoded/executed; its contents are decoded by the control unit to determine the operation to perform. (3) Memory Address Register (MAR): holds the address of the memory location to be accessed (read or written); it connects the CPU to the memory address bus. (4) Memory Data Register (MDR) / Memory Buffer Register (MBR): holds the data read from or to be written to memory; it acts as a buffer between the CPU and memory data bus. (5) Accumulator (AC): a general-purpose register used to hold operands and results of arithmetic and logic operations; in simple architectures it is implicitly used by many instructions. (6) General-Purpose Registers (GPRs): a set of registers (e.g., R0–R7) used for temporary storage of data, operands, intermediate results, and addresses; they reduce memory accesses and support flexible programming. (7) Status/Flag Register (Program Status Word, PSW): contains condition flags (zero, carry, sign, overflow) and control bits (interrupt enable, mode) that record the outcome of operations and the CPU state. (8) Stack Pointer (SP): holds the address of the top of the stack in memory, used for subroutine calls, returns, and interrupt handling. (9) Index Register: holds an offset used in address calculation for indexed addressing, facilitating array and table access. (10) Instruction Decoder/Control registers (e.g., control unit registers): hold decoded control signals and sequencing information; sometimes a separate instruction address register or temporary registers are used in the datapath. In summary, registers provide the CPU with fast temporary storage for instructions, data, addresses, and status information, enabling the fetch-decode-execute cycle.

Marking points:

• *1 mark: Program Counter (PC) – holds address of next instruction; updated after fetch.*

• *1 mark: Instruction Register (IR) – holds current instruction for decoding/execution.*

• *1 mark: Memory Address Register (MAR) – holds address of memory location to be accessed.*

• *1 mark: Memory Data Register (MDR/MBR) – buffers data transferred to/from memory.*

• *1 mark: Accumulator (AC) – holds operands and results of arithmetic/logic operations.*

• *1 mark: General-Purpose Registers – temporary storage for data, operands, intermediate results.*

• *1 mark: Status/Flag Register (PSW) – holds condition flags and CPU state bits.*

• *1 mark: Stack Pointer (SP) – points to top of stack for subroutine/interrupt handling.*

• *1 mark: Index Register – holds offset for indexed addressing/address calculation.*

• *1 mark: Any other relevant register (e.g., instruction decoder/control register, temporary register) with correct function, or overall clarity and completeness.*

Model confidence: high.

Model note: The question does not specify a particular architecture; the answer assumes a generic/typical CPU with common registers. Some architectures may name or organize registers differently (e.g., no dedicated accumulator, multiple MAR/MDR). If the original paper included a figure or register list, the answer should be matched to that list; here only the question text is available.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION 11 (20 marks)

Source: CSC429-2013-12, page [6]

Unofficial model answer:

Subpart (a) — One-address instructions. In a one-address machine every arithmetic instruction implicitly uses an accumulator (here Register X), and the single operand is a memory location. The equation in Figure 4 is taken to be X = (A + B) × (C − D) ÷ (E + F); the figure itself was supplied only as a cropped image, so this expression is an assumption. Program: `LOAD A` (X ← A); `ADD B` (X ← X + B = A + B); `STORE T1` (save A + B); `LOAD C` (X ← C); `SUB D` (X ← C − D); `STORE T2` (save C − D); `LOAD T1` (X ← A + B); `MUL T2` (X ← (A+B)(C−D)); `STORE T3`; `LOAD E` (X ← E); `ADD F` (X ← E + F); `STORE T4`; `LOAD T3` (X ← numerator); `DIV T4` (X ← (A+B)(C−D)/(E+F)); `STORE X` (final answer in X). Temporary memory locations T1–T4 are needed because only one operand can be addressed per instruction.

Subpart (b) — Two-address instructions. A two-address instruction has the form OP (X ← Y op Z) or OP (X ← X op Y), destroying one source. Using the register-to-register forms from Table 2: `MOVE X, A` (X ← A); `ADD X, B` (X ← A + B); `MOVE Y, C` (Y ← C); `SUB Y, D` (Y ← C − D); `MUL X, Y` (X ← (A+B)(C−D)); `MOVE Z, E` (Z ← E); `ADD Z, F` (Z ← E + F); `DIV X, Z` (X ← (A+B)(C−D)/(E+F)). The final result is already in X, so no STORE is required. Only the temporary registers X, Y and Z are used; A–F are never overwritten.

Subpart (c) — Three-address instructions. A three-address instruction specifies two sources and a distinct destination, OP (X ← Y op Z), so no intermediate results need to be saved and no source is destroyed. Program: `ADD X, A, B` (X ← A + B); `SUB Y, C, D` (Y ← C − D); `MUL X, X, Y` (X ← (A+B)(C−D)); `ADD Z, E, F` (Z ← E + F); `DIV X, X, Z` (X ← (A+B)(C−D)/(E+F)). The answer ends in X as required. This is the shortest program, illustrating that three-address format minimises instruction count at the cost of a longer instruction encoding.

Marking points:

• 2 marks: correct identification/statement of the expression to be computed from Figure 4 (e.g. X = (A+B)(C−D)/(E+F))

• 6 marks: one-address program — correct use of LOAD/STORE/ADD/SUB/MUL/DIV with accumulator X, correct temporary storage of partial results, correct order of operations, final result in X (deduct for missing STORE or wrong operand order)

• 5 marks: two-address program — correct MOVE/ADD/SUB/MUL/DIV (X←Y op Z or X←X op Y) forms, correct use of temporary registers X, Y, Z, A–F unmodified, final result in X

• 5 marks: three-address program — correct OP (X←Y op Z) form, minimal instruction sequence, A–F unmodified, final result in X

• 2 marks: correct assembly notation and explanatory comments throughout; no modification of registers A–F

Model confidence: low.

Model note: Figure 4 was supplied only as a cropped image with no readable expression; the equation X = (A+B)(C−D)/(E+F) is assumed from typical past-paper style and must be checked against the actual figure. Table 2 text is garbled (e.g. 'MOVE (X e Y)', 'ADD (X k.. X + Y)'), so instruction mnemonics/operand orders are interpreted as standard one-, two- and three-address forms. This is an unofficial model answer, not an official UTM marking scheme.

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA